Authored by: Arpit Raj, LNMIIT Jaipur
Date: Tuesday, October 27, 2026
© 2026 Arpit Raj. All rights reserved.

C++ Templates, Overloading & Binding

Constructor Overloading

-> a type of function overloading, define multiple constructors in same class with different parameter allowing objects to be initialized in different ways.

Eg

```
class Student {
public:
    int age;
    string name;

    Student(string n) : name(n), age(0) { }
    Student(int a, string n) : age(a), name(n) { }
};
```

now

```
Student s1("Aadya");
Student s2("Arpit", 20);
```

both are allowed.
Aadya is created with `0` age.

- diff no of parameters, diff parameter type, diff parameter order are **valid** differentiators.
- BUT different parameter name or same parameter but different return type are **not valid** differs.

Using Constructors with Default Arguments

```
Student(string n = "Aadya", int a = 0) {
    name = n;
    age = a;
}
```

allows filling with default value if argument not provided.

Introduction to Templates

Templates -> need of templates:-

```
int getMax(int a, int b) {
    if (a > b) return a;
    else return b;
}

double getMax(double a, double b) {
    if (a > b) return a;
    else return b;
}
```

repetitive logic for different function where only datatype changes.

better

```
template <typename T>
T getMax(T a, T b) {
    if (a > b) return a;
    else return b;
}
```

- # types must support operations used.
- # here `T = int, double` supports `>`
- # `getMax(10, 7.8)` -> Fails as it cant deduce `T`.
  - however `getMax<double>(10, 7.8)` works because of explicit conversion.

templates gives a reusable blueprint and useful when same logic works across types.

Class Templates

templates with classes

```
template <typename T>
class Aadya {
public:
    T value;
    Aadya(T v) {
        this->value = v;
    }
};

Aadya<int> a1(100);
Aadya<string> a2("Hello");
```

(Note pointing to `<int>` and `<string>`): `< >` of template typenp specified when called these.

Multiple Types in Templates

```
template <typename T, typename U>
class Aadya {
public:
    T first;
    U second;
    Aadya(T f, U s) {
        this->first = f;
        this->second = s;
    }
};
```

Q

```
Aadya() {
    this->value = T();
}
Aadya(T v) {
    this->value = v;
}
```

now if i call

```
Box<int> b1;
Box<int> b2(50);
```

then `b1` is initialized with `int()` (0). `b2` is with 50.

© Arpit Raj, LNMIIT Jaipur

Template Specialization (Trick Question)

⚠️ TRICK QUEST: TEMPLATE SPECIALIZATION

```
template <typename T>
class Aadya {
public:
    void display(T value) { cout << value << "\n"; }
};

template <> // -> cant write T here
class Aadya<string> {
public:
    void display(string value) {
        cout << "|" << value << "|\n";
    }
};
```

(Note pointing to `template <>`): string type will forcefully go here.
now `Aadya<string> obj; obj.display("Hi Baabloo");`
O/p -> |Hi Baabloo|

💡 Overload vs Template Resolution

```
void show(T x) { cout << "template" << endl; }
void show(int x) { cout << "normal" << endl; }
```

now
`show(10)` // -> normal, because exact match
`show(9.4)` // -> template, because deduced

Partial specialization is supported for class templates, not function templates.

Early vs Late Binding

```
Shape *s = new Circle();
// or
Circle c;
Shape *s = &c;
```

then without any virtualization if we call `s->draw()`
then draw of shape is called, because compiler runs line by line, it sees an object of shape class is called, so it calls draw of shape, this is early binding.

BINDING TYPES SUMMARY

- **Early Binding:** The compiler resolves the function call at compile time based on the pointer/reference type (static type). e.g., calling `s->draw()` without `virtual`.
- **Late Binding:** But if `virtual` keyword is used it defers the binding decision to runtime, this is called Late Binding.
  - now `s->draw()` is a polymorphic because it calls relevant draw depending on derived object (final overrider).

© Arpit Raj, LNMIIT Jaipur